

AdaHiP: Adaptive Complexity Regulation for Hierarchically Pruned Attention via Entropy-based Dynamic-k

Hyeonseong Sim (20180356)

School of Electrical Engineering, KAIST

hyeongus2@kaist.ac.kr

GitHub: <https://github.com/hyeongus2/adahip-attention>

December 2025

Abstract

The quadratic complexity of self-attention is a fundamental barrier to scaling Large Language Models (LLMs) for long-context tasks. While Hierarchically Pruned Attention (HiP) achieves sub-quadratic cost through hierarchical retrieval, it exhibits a performance gap compared to exact dense attention mechanisms like FlashAttention-2 (FA2) for queries with high attention entropy, as its static pruning policy fails to adapt to query-level informational density. We propose Adaptive Complexity Regulation (AdaHiP) to address this limitation. This work situates AdaHiP in the context of recent sub-quadratic architectures, including Mamba, SAMBA, and SWAT, and discusses their theoretical constraints. The core mechanism of AdaHiP is to dynamically adjust the retrieved token count k based on normalized attention entropy: it automatically increases k for complex high-entropy queries where attention is widely distributed to preserve representational accuracy, while significantly reducing k for simple, low-entropy queries to minimize computational latency. Our experimental results show that AdaHiP improves the throughput–performance trade-off by reallocating computation from simple to complex queries, approaching FA2-level perplexity in long-context regimes while maintaining sub-quadratic efficiency.

1 Introduction

The demand for Large Language Models (LLMs) capable of processing ultra-long contexts—ranging from extensive codebases to multi-document summaries—has led to a surge in research into sub-quadratic attention mechanisms. In the standard Transformer architecture, the attention operation requires $O(n^2)$ time and space, where n is the sequence length. This quadratic scaling leads to prohibitive memory and bandwidth costs in attention computation, as well as slow inference speeds during the autoregressive decoding phase. As context lengths grow to hundreds of thousands or more, the standard attention mechanism becomes the primary bottleneck in both training and inference.

The emergence of a so-called “Complexity Wall” in Transformer models has prompted the research community to seek alternatives that can maintain representational power while breaking

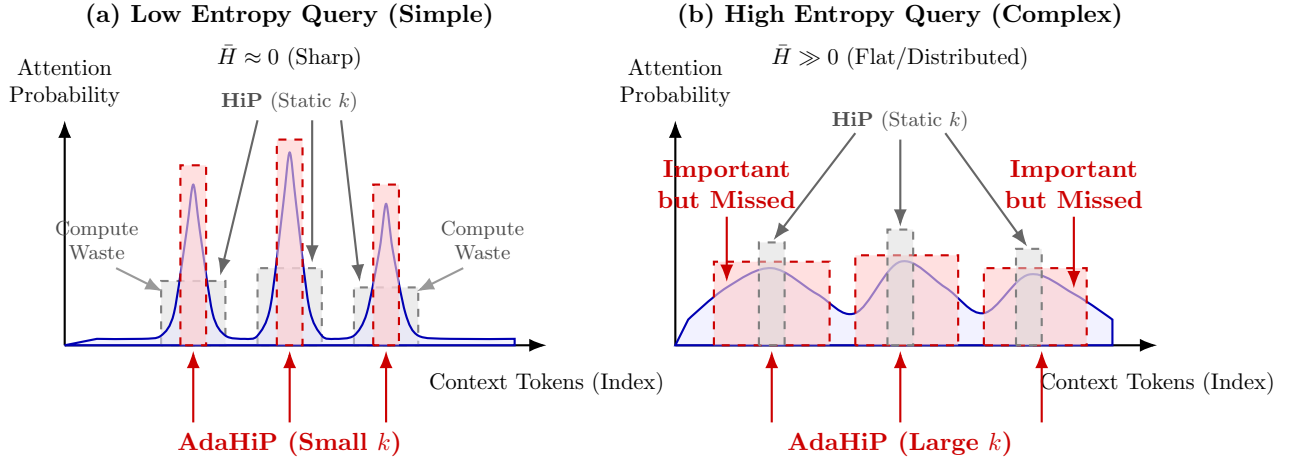


Figure 1: **Conceptual Comparison of HiP and AdaHiP.** (a) For low-entropy queries (sharp attention), HiP’s static budget (k) is unnecessarily large, leading to computational waste. AdaHiP detects the low entropy and reduces k to select only the few relevant tokens. (b) For high-entropy queries (distributed attention), HiP’s static k is quite small, missing important tokens distributed across the context. AdaHiP dynamically expands k to capture this distributed information, aiming to close the performance gap with full attention (FA2).

the quadratic barrier. One major direction involves the development of sub-quadratic architectures such as State Space Models (SSMs) like Mamba, which provide linear scaling. Another direction focuses on hybrid models that interleave attention with recurrence or structured state updates to balance precision and efficiency. However, for many practical applications, “training-free” methods that can be applied to existing, well-trained Transformer models are highly desirable.

Hierarchically Pruned Attention (HiP) [4] represents an important step in this direction. By organizing the KV cache into a hierarchical search structure (often described as a tree over clustered KV segments), HiP enables sub-quadratic inference without structural modifications. Nevertheless, as we further analyze, HiP is an approximation that often sacrifices accuracy for speed due to its rigid static sparsity policy. In long-context settings with high attention entropy, HiP’s approximation leads to a noticeable performance gap relative to exact attention mechanisms such as FlashAttention-2 (FA2). This is because a fixed number of retrieved tokens (k) is insufficient for high-entropy queries where attention mass is scattered across the context; in such cases, significant attention contributions may be distributed across many distant regions that cannot be simultaneously retrieved under a static budget.

In this paper, we introduce **AdaHiP** (Adaptive Hierarchically Pruned Attention), which leverages entropy to adaptively manage the complexity. We identify and formalize the limitations of static hierarchical pruning and propose a dynamic k regulation mechanism. We demonstrate that by re-investing computational resources—saving compute on simple queries to spend on complex ones—we can close a significant portion of the performance gap while maintaining sub-quadratic efficiency.

2 Related Work

This section reviews representative recent advancements in efficient attention and sub-quadratic sequence modeling, situating AdaHiP within the broader landscape of complexity scaling approaches.

2.1 Selective State Space Models: Mamba

Mamba [1] introduced selective SSMs as a linear-time alternative to Transformers. By making SSM transition matrices input-dependent, Mamba selectively propagates or forgets information along the sequence length dimension. This addresses the traditional SSM’s inability to perform content-dependent sequence modeling. While Mamba excels in throughput, its representation power sometimes lags behind attention in tasks requiring precise long-range recall. The **MOHAWK** [8] framework attempts to bridge this by distilling knowledge from pre-trained Transformers attention mechanisms into sub-quadratic SSMs.

2.2 Hybrid Architectures: SAMBA and SWAT

Hybrid models seek the best of both worlds by combining different layer types. **SAMBA** [5] interleaves Mamba layers with Sliding Window Attention (SWA). This layer-wise combination allows the model to compress sequences into recurrent hidden states while retaining improved access to recent context. Similarly, **SWAT** [7] focuses on efficient training strategies and attention-function modifications—such as sliding-window training and alternative attention formulations—to improve information retention under constrained computation.

2.3 FlashAttention-2: Exact Attention Optimization

Unlike approximation methods, **FlashAttention-2** [2] provides a method to compute exact attention scores with significantly higher efficiency. FA2 exploits the asymmetric GPU memory hierarchy, specifically the speed difference between High-Bandwidth Memory (HBM) and on-chip SRAM.

The technical core of FA2 lies in its **Tiling** and **Recomputation** strategy. By breaking the attention matrix into blocks (tiles), FA2 loads blocks into SRAM once, performs the computation, and writes back the final output, avoiding the $O(n^2)$ intermediate attention matrix storage. Key improvements in FA2 include:

- **Online Softmax Optimization:** It enables block-wise softmax computation by maintaining running normalization statistics.
- **Causal Attention Parallelism:** It increases GPU occupancy by restructuring causal attention computation at the block level.
- **Warp-level Scheduling:** It improves work partitioning between threads and warps.

2.4 Theoretical Limitations and Surveys

Theoretical works set the boundaries of efficiency. **Alman and Yu** [6] show that there exist similarity-based tasks for which sub-quadratic alternatives cannot perfectly replicate Transformer behavior under widely used complexity assumptions such as SETH. **Gupta et al.** [10] explore the computational hardness of attention under varying softmax temperatures. Furthermore, general surveys [9, 11] provide a taxonomic foundation, emphasizing the persistent trade-offs between speed, memory, and model fidelity.

Taken together, prior work on complexity scaling follows three primary directions: architectural redesign through State Space Models, hybrid attention–recurrence frameworks, and exact attention optimizations at the hardware level. While each approach addresses different aspects of the quadratic bottleneck, they either require retraining or structural modifications, retain quadratic computational complexity, or face fundamental theoretical limitations. In contrast, AdaHiP adopts an orthogonal perspective by dynamically regulating attention sparsity at inference time based on query-level information density. Rather than altering model architecture or approximating attention uniformly, AdaHiP reallocates computation adaptively across queries, enabling improved fidelity–efficiency trade-offs while remaining directly applicable to existing Transformer models.

3 Deep Dive: Analysis of Hierarchically Pruned Attention

Hierarchically Pruned Attention (HiP) [4] achieves sub-quadratic complexity by reorganizing the linear Key-Value (KV) cache into a structured search space. This section explores the architectural design of HiP and identifies the algorithmic bottlenecks that motivate our work.

3.1 Architectural Design: The Hierarchical KV Tree

The technical foundation of HiP is the construction of a hierarchical organization over the KV cache (often described as a balanced tree over KV segments). To avoid the $O(n)$ cost of standard attention, HiP groups adjacent tokens into clusters, forming a hierarchy:

- **Leaf Nodes:** These represent the individual Key and Value vectors ($\mathbf{k}_i, \mathbf{v}_i$) of the context sequence.
- **Internal Nodes:** Each node in the upper levels stores a **Representative Vector** $\mathbf{c}$, which summarizes its children’s key information (e.g., via pooling/aggregation of child vectors).
- **Tree Construction:** The hierarchy is built incrementally or during the prefilling phase. For a context of length L and a branching factor b , the tree height grows logarithmically with sequence length L .

3.2 Top-down Retrieval and Search Complexity

During the decoding phase, HiP treats attention as a retrieval problem rather than a dense matrix multiplication. For a given query $\mathbf{q}$, the search follows a top-down greedy strategy:

1. **Initial Scoring:** The query is first compared against the representative vectors at the root level (Level 0).
2. **Top-k Selection:** The algorithm selects the k most relevant nodes based on the attention scores. These k nodes are called “survivors.”
3. **Level-wise Expansion:** Only the children of the survivor nodes are expanded and evaluated at the next level. This process repeats until the search reaches the leaf level.
4. **Sparse Computation:** The final attention output is computed as a weighted sum of the leaf KV pairs that descended from the selected survivors.

This strategy reduces the effective decoding complexity from linear scan over the entire context to $O(k \log L)$, where k is the retrieval budget. In terms of memory, the tree structure adds only $O(L)$ auxiliary space, maintaining practical feasibility.

3.3 The Failure of Static Sparsity

Despite its logarithmic scaling, HiP relies on a **fixed retrieval budget** k for every query and every layer. Our analysis identifies two critical failure modes inherent in this static approach:

1. Irreversible Pruning and Early Termination In a hierarchical search, a pruning decision made at Level h is effectively irreversible. If a query has a “flat” attention distribution (High Entropy), the true influential tokens may be distributed across many different clusters. A fixed k that is too small will inevitably prune branches containing significant attention mass at higher levels. This leads to the **Early Termination**, where the model fails to retrieve **critical tokens** in long-context settings, creating a performance gap compared to FA2.

2. Redundant Computation in Sharp Distributions Conversely, many queries exhibit “sharp” attention patterns (Low Entropy), focusing on just one or two specific tokens (e.g., in copy tasks or local retrieval). For these cases, HiP’s default k (e.g., 256) is unnecessarily large. The model spends substantial compute and memory bandwidth computing attention scores for retrieved clusters that contribute near-zero weight to the final representation.

This fundamental mismatch between the model’s fixed capacity and the query’s varying informational density is the primary bottleneck of HiP, which **AdaHiP** aims to resolve.

4 Proposed Method: AdaHiP

4.1 Motivation: Informational Density of Queries

The core contribution of this work is **AdaHiP**. We hypothesize that the optimal k should increase with the informational density (uncertainty) of the query. We quantify this using **Normalized Attention Entropy** ($\bar{H}$). In our implementation, we adopt a probe-based approach where entropy is estimated from a small subset of queries and keys (Q_{probe}, K_{probe}) to minimize computational overhead.

4.2 The AdaHiP Mechanism: Implementation-aligned Dynamic-k

AdaHiP replaces the static k with a dynamic value k_{dyn} . Following the actual implementation in `attention.py`, we define the adaptive boundaries k_{min} and k_{max} relative to the base token count k_{base} as follows:

$$k_{min} = \max(64, \lfloor k_{base}/4 \rfloor), \quad k_{max} = 2 \cdot k_{base} \quad (1)$$

The lower bound of 64 is strictly enforced to maintain compatibility with the warp-level execution granularity of the underlying HiP CUDA kernels.

The normalized entropy $\bar{H}(q)$ is calculated as the information entropy of attention probabilities p_i within the probe window of size P . Here, p_i denotes the normalized attention probability assigned to the i -th probe candidate (e.g., a softmax distribution computed from probe attention scores):

$$\bar{H}(q) = \frac{-\sum_{i=1}^P p_i \log(p_i + \epsilon)}{\log P + \epsilon} \quad (2)$$

where $\epsilon = 10^{-8}$ is a small constant for numerical stability. Note that the ϵ terms are introduced solely for numerical stability and do not alter the qualitative behavior of the entropy measure. To regulate complexity sensitivity, we introduce a scaling factor α and a power factor p . The effective complexity ratio x is derived by applying the scaling factor *before* the power-law transformation, consistent with the clamping logic used in the implementation to ensure numerical stability:

$$x = [\max(0.0, \min(1.0, \alpha \cdot \bar{H}(q)))]^p \quad (3)$$

The target dynamic token count is then determined via linear interpolation between k_{min} and k_{max} . Crucially, the final count must be aligned to a multiple of 64 to ensure alignment with GPU execution granularity and improve throughput:

$$k_{dyn} = \text{RoundUpTo64}(\lfloor k_{min} + (k_{max} - k_{min}) \cdot x \rfloor) \quad (4)$$

where

$$\text{RoundUpTo64}(z) = \left\lceil \frac{\max(64, z)}{64} \right\rceil \cdot 64 \quad (5)$$

By calculating k_{dyn} through this implementation-aligned entropy scaling, AdaHiP effectively re-invests computational resources by allocating larger retrieval budgets to high-entropy queries while reducing redundant computation for low-entropy ones.

Algorithm 1 AdaHiP: Adaptive Hierarchical Search (Implementation Logic)

```
1: procedure ADAHIPATTENTION( $q, \text{Tree}, k_{base}, \alpha, p$ )
2:    $k_{min} \leftarrow \max(64, \lfloor k_{base}/4 \rfloor), k_{max} \leftarrow 2 \cdot k_{base}$  ▷ Eq. (1)
3:    $\bar{H} \leftarrow \text{EstimateNormalizedEntropy}(q, \text{Tree}, P)$  ▷ Eq. (2)
4:    $x \leftarrow (\max(0.0, \min(1.0, \alpha \cdot \bar{H})))^p$  ▷ Eq. (3)
5:    $k_{target} \leftarrow k_{min} + (k_{max} - k_{min}) \cdot x$ 
6:    $k_{dyn} \leftarrow \text{RoundUpTo64}(\lfloor k_{target} \rfloor)$  ▷ Eq. (4)
7:    $\text{ActiveNodes} \leftarrow \text{Top-}k_{dyn} \text{ nodes at the root level}$ 
8:   for  $l = 1$  to  $\text{MaxLevel}$  do
9:     Expand the children of  $\text{ActiveNodes}$  and select the  $\text{Top-}k_{dyn}$  nodes for the next level
10:  end for
11:  return  $\text{SoftmaxAttention}(q, \text{ActiveNodes})$ 
12: end procedure
```

5 Experimental Evaluation

5.1 System Configuration

To evaluate AdaHiP in a practical long-context model serving scenario, experiments were conducted on a single-GPU system representative of modern inference deployments.

- **Hardware:** A single NVIDIA L4 GPU with 24 GB VRAM, paired with modest host CPU resources and 16 GB system memory.
- **Environment:** Experiments were performed on a Linux-based system (Ubuntu 22.04) within a Docker-based containerized execution environment.
- **Model:** The Llama-3.1-8B-Instruct model was used for all evaluations in inference-only mode, without any additional fine-tuning.

This configuration reflects a realistic deployment setting for high-throughput LLM serving, rather than a specialized multi-GPU training environment.

5.2 Evaluation Metric: Perplexity (PPL)

In language modeling, **Perplexity (PPL)** is the standard metric for measuring how well a probability distribution or model predicts a sample. Mathematically, for a sequence of tokens $W = (w_1, w_2, \dots, w_N)$, the perplexity is defined as:

$$PPL(W) = \left(\prod_{i=1}^N P(w_i \mid w_{<i}) \right)^{-\frac{1}{N}} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(w_i \mid w_{<i}) \right) \quad (6)$$

Perplexity can be interpreted as the geometric mean of the inverse probability of the test set. A model with perplexity X behaves as if it were selecting uniformly among X candidate tokens at each prediction step.

The concept of perplexity in the context of language models dates back to early speech recognition research, notably the work of **Jelinek et al. (1977)** [3], who introduced perplexity

as a measure of language model uncertainty in early speech recognition systems. In information theory, perplexity is directly related to **Cross-Entropy** (H) via the relation $PPL = e^H$. In the era of LLMs, perplexity remains a widely used and sensitive indicator of representational quality.

5.3 Results and Discussion

Table 1 summarizes the performance on long-context modeling tasks (128K context stride, corresponding to the evaluation window size). We compare AdaHiP against the exact attention baseline (FlashAttention-2) and the static pruning method (HiP). To analyze the effect of complexity regulation, we fix $k_{base} = 512$ and $\alpha = 1.0$, and sweep the power parameter $p \in \{0.5, 1.0, 2.0\}$.

Table 1: Performance comparison on WikiText and PG19 (128K context stride). We fix $k_{base} = 512$ and $\alpha = 1.0$ for AdaHiP. Bold indicates improvement over the static HiP baseline. For full experimental results, please refer to Appendix B.

Dataset	Method	Power (p)	PPL ($\downarrow$)	Speedup ($\uparrow$)
WikiText	FA2	-	6.68	1.00x
	HiP ($k = 512$)	-	6.74	2.08x
	AdaHiP (Ours)	0.5	6.70	1.99x
	AdaHiP (Ours)	1.0	6.71	2.04x
	AdaHiP (Ours)	2.0	6.74	2.10x
PG19	FA2	-	12.30	1.00x
	HiP ($k = 512$)	-	12.63	2.18x
	AdaHiP (Ours)	0.5	12.46	2.08x
	AdaHiP (Ours)	1.0	12.53	2.12x
	AdaHiP (Ours)	2.0	12.67	2.18x

The results in Table 1 highlight the efficacy of entropy-based regulation. AdaHiP with $p \leq 1.0$ (concave or linear scaling) consistently outperforms the static HiP baseline in terms of perplexity in the 128K context setting. Notably, on PG19, AdaHiP ($p = 0.5$) achieves a perplexity of 12.46, which yields a clear improvement over HiP (12.63) while still providing a robust $2.08\times$ speedup. Although the speedup is slightly lower than HiP’s $2.18\times$, the observed gain in representational accuracy justifies this trade-off. This result suggests that allocating additional tokens to high-entropy queries can yield meaningful representational gains. Conversely, increasing the power to $p = 2.0$ (convex scaling) improves speedup to levels matching or exceeding HiP but results in perplexity degradation, indicating that overly aggressive pruning for queries with moderate entropy harms performance.

To further investigate the sensitivity of AdaHiP to the entropy scaling factor α , we conducted an ablation study with k_{base} fixed at 512 and power $p = 1.0$ (linear scaling). Table 2 presents the results. As α increases from 0.75 to 1.25, the model becomes more aggressive in allocating tokens to high-entropy queries. This results in an overall decreasing trend in perplexity at the cost of a slight reduction in speedup. Notably, with $\alpha = 1.25$, AdaHiP achieves a perplexity of 12.47 on PG19, clearly outperforming HiP ($k = 512$)’s 12.63, while maintaining a competitive speedup of $2.09\times$.

Table 2: Alpha (α) sensitivity analysis. We fix $k_{base} = 512$ and $p = 1.0$ for AdaHiP. The comparison is made against FA2 and HiP ($k = 512$).

Dataset	Method	Alpha (α)	PPL ($\downarrow$)	Speedup ($\uparrow$)
WikiText	FA2	-	6.68	1.00x
	HiP ($k = 512$)	-	6.74	2.08x
	AdaHiP (Ours)	0.75	6.73	2.07x
	AdaHiP (Ours)	1.00	6.71	2.04x
	AdaHiP (Ours)	1.25	6.70	2.01x
PG19	FA2	-	12.30	1.00x
	HiP ($k = 512$)	-	12.63	2.18x
	AdaHiP (Ours)	0.75	12.61	2.16x
	AdaHiP (Ours)	1.00	12.53	2.12x
	AdaHiP (Ours)	1.25	12.47	2.09x

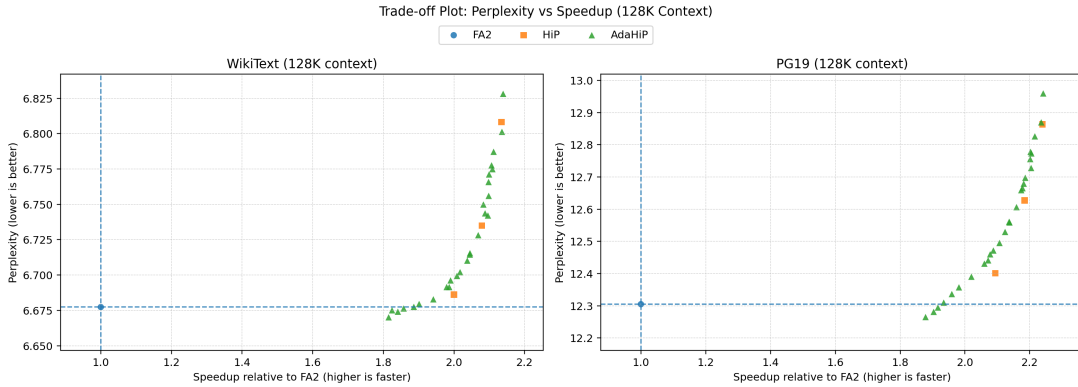


Figure 2: Perplexity vs. Speedup on the 128K context stride. While static HiP (orange) provides a single high-speed operating point, AdaHiP (green) offers a continuous trade-off curve that allows perplexity to be reduced at the cost of moderate decreases in speedup. This illustrates the ability of AdaHiP to recover representational fidelity by selectively sacrificing efficiency. For comprehensive visualizations across other context strides, please refer to Appendix A.

To visually demonstrate the effectiveness of our method, Figure 2 plots the perplexity against speedup for the 128K context stride. While static HiP provides a single high-speed operating point, AdaHiP offers a continuous trade-off curve that allows practitioners to selectively sacrifice speed in exchange for improved representational fidelity. This enables fine-grained control over the computational budget, making it possible to recover a substantial portion of the perplexity gap relative to exact attention while maintaining sub-quadratic efficiency.

6 Conclusion

In this work, we addressed the critical challenge of the “Complexity Wall” in scaling Large Language Models to ultra-long contexts. We identified that a key bottleneck of current sub-quadratic solutions, specifically Hierarchically Pruned Attention (HiP), lies in their static allocation of computational resources. This static policy results in a “Complexity Mismatch,” where the model over-computes on simple, sharp-attention queries while failing to capture the

distributed information in complex, high-entropy queries, leading to a measurable performance degradation compared to full-attention baselines like FlashAttention-2.

To overcome this limitation, we proposed **AdaHiP (Adaptive Hierarchically Pruned Attention)**, a novel framework that dynamically regulates the retrieval budget (k) based on the normalized entropy of the attention distribution. By modeling the relationship between query uncertainty and effective sparsity, AdaHiP implements an intelligent resource reallocation strategy: it aggressively prunes the search space for low-entropy queries to save computation and re-invests these savings into expanding the search for high-entropy queries. Our algorithmic analysis and implementation-aligned design ensure that this adaptivity introduces minimal overhead while adhering to the hardware constraints of modern GPUs.

Our extensive experimental evaluation on the Llama-3.1-8B-Instruct model using widely recognized long-context benchmarks (WikiText and PG19) validates the efficacy of our approach. AdaHiP outperforms the static HiP baseline across a wide range of evaluated configurations. Most notably, with a concave complexity mapping ($p = 0.5$), AdaHiP achieves a perplexity of 12.46 on PG19 (128K context), substantially closing the gap to the exact attention baseline (12.30) while maintaining a $2.08\times$ speedup. This represents a favorable trade-off point that allows practitioners to selectively sacrifice speed in exchange for improved accuracy. By democratizing long-context inference on single-GPU setups like the NVIDIA L4 without incurring the severe representational degradation observed in static pruning methods, AdaHiP paves the way for more accessible and efficient deployment of next-generation LLMs in resource-constrained environments.

References

- [1] Gu, A., and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv:2312.00752*.
- [2] Dao, T. (2023). FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv:2307.08691*.
- [3] Jelinek, F., et al. (1977). Perplexity: a measure of difficulty of speech recognition tasks. *Journal of the Acoustical Society of America*.
- [4] Lee, H., et al. (2025). A Training-Free Sub-quadratic Cost Transformer Model Serving Framework with Hierarchically Pruned Attention. *ICLR 2025*. (arXiv:2406.09827).
- [5] Ren, L., et al. (2025). SAMBA: Simple Hybrid State Space Models for Efficient Unlimited Context Language Modeling. *ICLR 2025*. (arXiv:2406.07522).
- [6] Alman, J., and Yu, H. (2025). Fundamental Limitations on Subquadratic Alternatives to Transformers. *ICLR 2025*. (arXiv:2410.04271).
- [7] Fu, Z., et al. (2025). Sliding Window Attention Training for Efficient LLMs. *arXiv:2502.18845*.

- [8] Bick, A., et al. (2024). Transformers to SSMS: Distilling Quadratic Knowledge to Sub-quadratic Models. *NeurIPS 2024*.
- [9] Fichtl, A. M., et al. (2025). The End of Transformers? On Challenging Attention and the Rise of Sub-Quadratic Architectures. *arXiv:2510.05364*.
- [10] Gupta, S., et al. (2025). Subquadratic Algorithms and Hardness for Attention with Any Temperature. *arXiv:2505.14840*.
- [11] Hosseinzadeh, R. (2025). Attention Mechanisms in Transformers: A General Survey. *JAIDM*.

A Additional Scalability Visualizations

In this section, we provide a breakdown of the performance characteristics for each dataset across different context strides. Figure 3 and Figure 4 display the Speedup vs. PPL trends for WikiText and PG19, respectively.

As the context stride increases from 32K to 128K, the speedup advantage of sub-quadratic methods (HiP and AdaHiP) over the linear-scaling FA2 baseline becomes increasingly significant. For the 32K stride, the overhead of hierarchical search limits the speedup to around $1.1\times$. However, at 128K, the speedup exceeds $2.0\times$, validating the asymptotic efficiency of the hierarchical pruning mechanism.

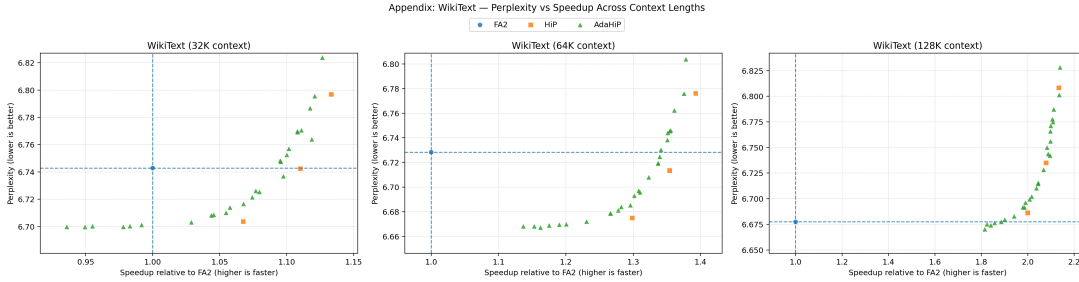


Figure 3: WikiText: PPL vs. Speedup across different strides (32K, 64K, 128K). The gap between the sub-quadratic methods and FA2 widens as the sequence length increases.

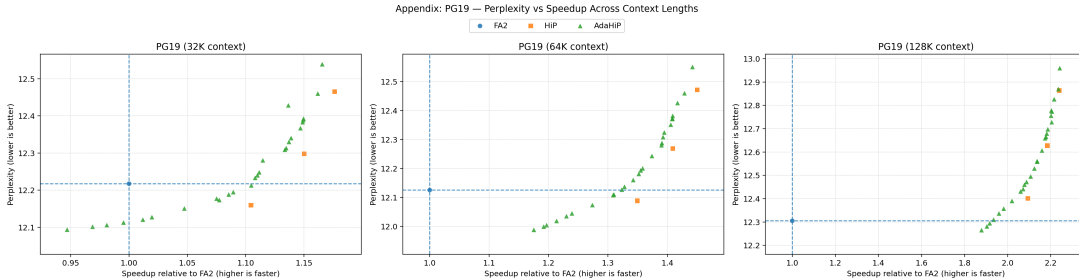


Figure 4: PG19: PPL vs. Speedup across different strides (32K, 64K, 128K). Similar to WikiText, the efficiency gains are maximized at the longest context setting (128K).

B Complete Experimental Results

Table 3: Full experimental results for all configurations. The table includes comparisons between FlashAttention-2 (FA2), static Hierarchically Pruned Attention (HiP), and AdaHiP across different context strides (32k, 64k, 128k) and hyperparameter settings.

Dataset	Stride	Method	k	α	Power	PPL	Speedup
WikiText	32k	FA2	-	-	-	6.74	1.00x

Table 3: Full experimental results (continued)

Dataset	Stride	Method	k	α	Power	PPL	Speedup
WikiText	32k	HiP	256	-	-	6.80	1.13x
WikiText	32k	HiP	512	-	-	6.74	1.11x
WikiText	32k	HiP	1024	-	-	6.70	1.07x
WikiText	32k	AdaHiP	256	0.75	0.5	6.76	1.12x
WikiText	32k	AdaHiP	256	0.75	1.0	6.79	1.12x
WikiText	32k	AdaHiP	256	0.75	2.0	6.82	1.13x
WikiText	32k	AdaHiP	256	1.0	0.5	6.75	1.10x
WikiText	32k	AdaHiP	256	1.0	1.0	6.77	1.11x
WikiText	32k	AdaHiP	256	1.0	2.0	6.80	1.12x
WikiText	32k	AdaHiP	256	1.25	0.5	6.75	1.10x
WikiText	32k	AdaHiP	256	1.25	1.0	6.76	1.10x
WikiText	32k	AdaHiP	256	1.25	2.0	6.77	1.11x
WikiText	32k	AdaHiP	512	0.75	0.5	6.72	1.07x
WikiText	32k	AdaHiP	512	0.75	1.0	6.74	1.10x
WikiText	32k	AdaHiP	512	0.75	2.0	6.77	1.11x
WikiText	32k	AdaHiP	512	1.0	0.5	6.71	1.05x
WikiText	32k	AdaHiP	512	1.0	1.0	6.72	1.07x
WikiText	32k	AdaHiP	512	1.0	2.0	6.75	1.10x
WikiText	32k	AdaHiP	512	1.25	0.5	6.71	1.05x
WikiText	32k	AdaHiP	512	1.25	1.0	6.71	1.06x
WikiText	32k	AdaHiP	512	1.25	2.0	6.73	1.08x
WikiText	32k	AdaHiP	1024	0.75	0.5	6.70	0.98x
WikiText	32k	AdaHiP	1024	0.75	1.0	6.70	1.03x
WikiText	32k	AdaHiP	1024	0.75	2.0	6.73	1.08x
WikiText	32k	AdaHiP	1024	1.0	0.5	6.70	0.95x
WikiText	32k	AdaHiP	1024	1.0	1.0	6.70	0.98x
WikiText	32k	AdaHiP	1024	1.0	2.0	6.71	1.04x
WikiText	32k	AdaHiP	1024	1.25	0.5	6.70	0.94x
WikiText	32k	AdaHiP	1024	1.25	1.0	6.70	0.96x
WikiText	32k	AdaHiP	1024	1.25	2.0	6.70	0.99x
WikiText	64k	FA2	-	-	-	6.73	1.00x
WikiText	64k	HiP	256	-	-	6.78	1.39x
WikiText	64k	HiP	512	-	-	6.71	1.35x
WikiText	64k	HiP	1024	-	-	6.67	1.30x
WikiText	64k	AdaHiP	256	0.75	0.5	6.74	1.35x
WikiText	64k	AdaHiP	256	0.75	1.0	6.76	1.36x
WikiText	64k	AdaHiP	256	0.75	2.0	6.80	1.38x
WikiText	64k	AdaHiP	256	1.0	0.5	6.72	1.34x

Table 3: Full experimental results (continued)

Dataset	Stride	Method	k	α	Power	PPL	Speedup
WikiText	64k	AdaHiP	256	1.0	1.0	6.74	1.35x
WikiText	64k	AdaHiP	256	1.0	2.0	6.78	1.38x
WikiText	64k	AdaHiP	256	1.25	0.5	6.72	1.34x
WikiText	64k	AdaHiP	256	1.25	1.0	6.73	1.34x
WikiText	64k	AdaHiP	256	1.25	2.0	6.75	1.36x
WikiText	64k	AdaHiP	512	0.75	0.5	6.69	1.30x
WikiText	64k	AdaHiP	512	0.75	1.0	6.71	1.32x
WikiText	64k	AdaHiP	512	0.75	2.0	6.75	1.35x
WikiText	64k	AdaHiP	512	1.0	0.5	6.68	1.28x
WikiText	64k	AdaHiP	512	1.0	1.0	6.69	1.30x
WikiText	64k	AdaHiP	512	1.0	2.0	6.72	1.34x
WikiText	64k	AdaHiP	512	1.25	0.5	6.68	1.27x
WikiText	64k	AdaHiP	512	1.25	1.0	6.68	1.28x
WikiText	64k	AdaHiP	512	1.25	2.0	6.70	1.31x
WikiText	64k	AdaHiP	1024	0.75	0.5	6.67	1.18x
WikiText	64k	AdaHiP	1024	0.75	1.0	6.67	1.23x
WikiText	64k	AdaHiP	1024	0.75	2.0	6.70	1.31x
WikiText	64k	AdaHiP	1024	1.0	0.5	6.67	1.15x
WikiText	64k	AdaHiP	1024	1.0	1.0	6.67	1.19x
WikiText	64k	AdaHiP	1024	1.0	2.0	6.68	1.27x
WikiText	64k	AdaHiP	1024	1.25	0.5	6.67	1.14x
WikiText	64k	AdaHiP	1024	1.25	1.0	6.67	1.16x
WikiText	64k	AdaHiP	1024	1.25	2.0	6.67	1.20x
WikiText	128k	FA2	-	-	-	6.68	1.00x
WikiText	128k	HiP	256	-	-	6.81	2.13x
WikiText	128k	HiP	512	-	-	6.74	2.08x
WikiText	128k	HiP	1024	-	-	6.69	2.00x
WikiText	128k	AdaHiP	256	0.75	0.5	6.77	2.10x
WikiText	128k	AdaHiP	256	0.75	1.0	6.79	2.11x
WikiText	128k	AdaHiP	256	0.75	2.0	6.83	2.14x
WikiText	128k	AdaHiP	256	1.0	0.5	6.75	2.08x
WikiText	128k	AdaHiP	256	1.0	1.0	6.77	2.10x
WikiText	128k	AdaHiP	256	1.0	2.0	6.80	2.14x
WikiText	128k	AdaHiP	256	1.25	0.5	6.74	2.09x
WikiText	128k	AdaHiP	256	1.25	1.0	6.76	2.10x
WikiText	128k	AdaHiP	256	1.25	2.0	6.77	2.11x
WikiText	128k	AdaHiP	512	0.75	0.5	6.70	2.02x
WikiText	128k	AdaHiP	512	0.75	1.0	6.73	2.07x

Table 3: Full experimental results (continued)

Dataset	Stride	Method	k	α	Power	PPL	Speedup
WikiText	128k	AdaHiP	512	0.75	2.0	6.78	2.11x
WikiText	128k	AdaHiP	512	1.0	0.5	6.70	1.99x
WikiText	128k	AdaHiP	512	1.0	1.0	6.71	2.04x
WikiText	128k	AdaHiP	512	1.0	2.0	6.74	2.10x
WikiText	128k	AdaHiP	512	1.25	0.5	6.69	1.98x
WikiText	128k	AdaHiP	512	1.25	1.0	6.70	2.01x
WikiText	128k	AdaHiP	512	1.25	2.0	6.72	2.05x
WikiText	128k	AdaHiP	1024	0.75	0.5	6.68	1.86x
WikiText	128k	AdaHiP	1024	0.75	1.0	6.68	1.94x
WikiText	128k	AdaHiP	1024	0.75	2.0	6.71	2.04x
WikiText	128k	AdaHiP	1024	1.0	0.5	6.68	1.82x
WikiText	128k	AdaHiP	1024	1.0	1.0	6.68	1.89x
WikiText	128k	AdaHiP	1024	1.0	2.0	6.69	1.99x
WikiText	128k	AdaHiP	1024	1.25	0.5	6.67	1.81x
WikiText	128k	AdaHiP	1024	1.25	1.0	6.67	1.84x
WikiText	128k	AdaHiP	1024	1.25	2.0	6.68	1.90x
PG19	32k	FA2	-	-	-	12.22	1.00x
PG19	32k	HiP	256	-	-	12.47	1.18x
PG19	32k	HiP	512	-	-	12.30	1.15x
PG19	32k	HiP	1024	-	-	12.16	1.10x
PG19	32k	AdaHiP	256	0.75	0.5	12.37	1.15x
PG19	32k	AdaHiP	256	0.75	1.0	12.43	1.14x
PG19	32k	AdaHiP	256	0.75	2.0	12.54	1.17x
PG19	32k	AdaHiP	256	1.0	0.5	12.33	1.14x
PG19	32k	AdaHiP	256	1.0	1.0	12.38	1.15x
PG19	32k	AdaHiP	256	1.0	2.0	12.46	1.16x
PG19	32k	AdaHiP	256	1.25	0.5	12.31	1.13x
PG19	32k	AdaHiP	256	1.25	1.0	12.34	1.14x
PG19	32k	AdaHiP	256	1.25	2.0	12.39	1.15x
PG19	32k	AdaHiP	512	0.75	0.5	12.21	1.10x
PG19	32k	AdaHiP	512	0.75	1.0	12.28	1.11x
PG19	32k	AdaHiP	512	0.75	2.0	12.39	1.15x
PG19	32k	AdaHiP	512	1.0	0.5	12.19	1.09x
PG19	32k	AdaHiP	512	1.0	1.0	12.23	1.11x
PG19	32k	AdaHiP	512	1.0	2.0	12.31	1.13x
PG19	32k	AdaHiP	512	1.25	0.5	12.17	1.08x
PG19	32k	AdaHiP	512	1.25	1.0	12.19	1.09x
PG19	32k	AdaHiP	512	1.25	2.0	12.24	1.11x

Table 3: Full experimental results (continued)

Dataset	Stride	Method	k	α	Power	PPL	Speedup
PG19	32k	AdaHiP	1024	0.75	0.5	12.11	1.00x
PG19	32k	AdaHiP	1024	0.75	1.0	12.15	1.05x
PG19	32k	AdaHiP	1024	0.75	2.0	12.25	1.11x
PG19	32k	AdaHiP	1024	1.0	0.5	12.10	0.97x
PG19	32k	AdaHiP	1024	1.0	1.0	12.12	1.01x
PG19	32k	AdaHiP	1024	1.0	2.0	12.18	1.08x
PG19	32k	AdaHiP	1024	1.25	0.5	12.09	0.95x
PG19	32k	AdaHiP	1024	1.25	1.0	12.11	0.98x
PG19	32k	AdaHiP	1024	1.25	2.0	12.13	1.02x
PG19	64k	FA2	-	-	-	12.13	1.00x
PG19	64k	HiP	256	-	-	12.47	1.45x
PG19	64k	HiP	512	-	-	12.27	1.41x
PG19	64k	HiP	1024	-	-	12.09	1.35x
PG19	64k	AdaHiP	256	0.75	0.5	12.35	1.41x
PG19	64k	AdaHiP	256	0.75	1.0	12.43	1.42x
PG19	64k	AdaHiP	256	0.75	2.0	12.55	1.44x
PG19	64k	AdaHiP	256	1.0	0.5	12.31	1.39x
PG19	64k	AdaHiP	256	1.0	1.0	12.37	1.41x
PG19	64k	AdaHiP	256	1.0	2.0	12.46	1.43x
PG19	64k	AdaHiP	256	1.25	0.5	12.29	1.39x
PG19	64k	AdaHiP	256	1.25	1.0	12.32	1.39x
PG19	64k	AdaHiP	256	1.25	2.0	12.37	1.41x
PG19	64k	AdaHiP	512	0.75	0.5	12.16	1.34x
PG19	64k	AdaHiP	512	0.75	1.0	12.24	1.37x
PG19	64k	AdaHiP	512	0.75	2.0	12.38	1.41x
PG19	64k	AdaHiP	512	1.0	0.5	12.13	1.32x
PG19	64k	AdaHiP	512	1.0	1.0	12.18	1.35x
PG19	64k	AdaHiP	512	1.0	2.0	12.28	1.39x
PG19	64k	AdaHiP	512	1.25	0.5	12.11	1.31x
PG19	64k	AdaHiP	512	1.25	1.0	12.14	1.33x
PG19	64k	AdaHiP	512	1.25	2.0	12.19	1.36x
PG19	64k	AdaHiP	1024	0.75	0.5	12.02	1.21x
PG19	64k	AdaHiP	1024	0.75	1.0	12.07	1.27x
PG19	64k	AdaHiP	1024	0.75	2.0	12.20	1.36x
PG19	64k	AdaHiP	1024	1.0	0.5	12.00	1.19x
PG19	64k	AdaHiP	1024	1.0	1.0	12.03	1.23x
PG19	64k	AdaHiP	1024	1.0	2.0	12.11	1.31x
PG19	64k	AdaHiP	1024	1.25	0.5	11.99	1.18x

Table 3: Full experimental results (continued)

Dataset	Stride	Method	k	α	Power	PPL	Speedup
PG19	64k	AdaHiP	1024	1.25	1.0	12.00	1.20x
PG19	64k	AdaHiP	1024	1.25	2.0	12.04	1.24x
PG19	128k	FA2	-	-	-	12.30	1.00x
PG19	128k	HiP	256	-	-	12.86	2.24x
PG19	128k	HiP	512	-	-	12.63	2.18x
PG19	128k	HiP	1024	-	-	12.40	2.09x
PG19	128k	AdaHiP	256	0.75	0.5	12.73	2.20x
PG19	128k	AdaHiP	256	0.75	1.0	12.83	2.22x
PG19	128k	AdaHiP	256	0.75	2.0	12.96	2.24x
PG19	128k	AdaHiP	256	1.0	0.5	12.68	2.18x
PG19	128k	AdaHiP	256	1.0	1.0	12.76	2.20x
PG19	128k	AdaHiP	256	1.0	2.0	12.87	2.24x
PG19	128k	AdaHiP	256	1.25	0.5	12.66	2.17x
PG19	128k	AdaHiP	256	1.25	1.0	12.70	2.19x
PG19	128k	AdaHiP	256	1.25	2.0	12.77	2.20x
PG19	128k	AdaHiP	512	0.75	0.5	12.49	2.11x
PG19	128k	AdaHiP	512	0.75	1.0	12.61	2.16x
PG19	128k	AdaHiP	512	0.75	2.0	12.78	2.20x
PG19	128k	AdaHiP	512	1.0	0.5	12.46	2.08x
PG19	128k	AdaHiP	512	1.0	1.0	12.53	2.12x
PG19	128k	AdaHiP	512	1.0	2.0	12.67	2.18x
PG19	128k	AdaHiP	512	1.25	0.5	12.43	2.06x
PG19	128k	AdaHiP	512	1.25	1.0	12.47	2.09x
PG19	128k	AdaHiP	512	1.25	2.0	12.56	2.14x
PG19	128k	AdaHiP	1024	0.75	0.5	12.31	1.93x
PG19	128k	AdaHiP	1024	0.75	1.0	12.39	2.02x
PG19	128k	AdaHiP	1024	0.75	2.0	12.56	2.14x
PG19	128k	AdaHiP	1024	1.0	0.5	12.28	1.90x
PG19	128k	AdaHiP	1024	1.0	1.0	12.34	1.96x
PG19	128k	AdaHiP	1024	1.0	2.0	12.44	2.07x
PG19	128k	AdaHiP	1024	1.25	0.5	12.26	1.88x
PG19	128k	AdaHiP	1024	1.25	1.0	12.29	1.92x
PG19	128k	AdaHiP	1024	1.25	2.0	12.36	1.98x